

구현

Tool-Schema Patch 방식의 문제와 개선 방향

1. History 처리 단위

질문: History를 한 줄씩 처리하면 LLM 호출이 너무 많지 않은가? 기존 Hybrid는 어떻게 처리하는가?

현재 방식:

- Structured Hybrid는 최대 약 8K token 단위로 대화를 묶어 batch당 한 번 호출한다.
- Patch 방식은 한 줄을 하나의 turn으로 처리한다.
- Scenario 1에서는 2,438턴, 재시도 포함 2,456회 호출했다.

개선 방향:

20~50 turns 또는 2~4K tokens
→ LLM이 source turn이 표시된 순서형 Patch 목록 생성
→ source turn 순서대로 DB에 적용

초기에는 누락 방지를 위해 모든 batch를 LLM에 전달한다. 차량 관련 pre-filter는 성능 검증 후 선택적인 비용 최적화로 추가한다.

2. Memory 판단과 Patch 연산

질문: 매 turn마다 반드시 Patch를 수행하는가?

현재 방식:

LLM은 다음 중 하나를 반환한다.

- **ADD**: 새로운 설정 추가
- **UPDATE**: 기존 설정 변경
- **MERGE**: 중복 Memory 통합
- **DELETE**: 명시적인 폐기
- **NO-OP**: Memory가 아니면 `patches=[]`

즉 모든 turn에서 LLM은 호출되지만, 반드시 Memory가 생성되는 것은 아니다. 이번 실행에서는 2,438턴 중 39개의 Patch proposal만 생성됐다.

기존 Hybrid:

- LLM이 batch에서 장기 기억 가치가 있는 구조화된 후보를 추출한다.
- Memory가 없으면 빈 candidate 목록을 반환한다.
- 이후 중복·PII 등의 검사를 거쳐 저장한다.
- 명시적인 `ADD·UPDATE·MERGE·DELETE` 연산은 사용하지 않는다.

개선 방향:

Batch별로 여러 Patch를 생성하되 각 Patch에 `source_turn_id` 와 `sequence` 를 포함하여 시간 순서를 보존한다.

3. Patch LLM Context와 Tool Schema

질문: 111개 Tool은 어떻게 전달되며 Agent의 Tool discovery는 어떻게 동작하는가?

현재 Patch LLM Context:

```
고정 user/session 정보
+ 111개 Tool ontology 전체
+ 현재 active Memory 전체
+ 현재 source turn
```

각 Tool은 다음과 같은 JSON schema 형태로 전달된다.

```
tool_name, domain, action, topic, argument slots, type, enum, range
```

현재 Agent Tool discovery:

```
현재 질문 + 선택된 Memory
→ Agent LLM이 module 선택
→ list_module_tools(module) 호출
→ 해당 module의 Tool schema 로딩
→ 실제 Tool 호출
```

즉 module 선택은 Agent LLM 기반이고, 선택된 module의 Tool 목록 제공은 규칙 기반이다.

개선 방향:

- Gold Tool 고정은 원인 분석을 위한 Oracle 실험을 할 수 있도록 옵션 추가
 - 최종 방식은 keyword, Tool-description embedding, 선택적 LLM classifier를 결합
 - 높은 Recall을 유지하도록 여러 Tool/domain 후보를 남김
 - 관련 Tool schema와 active record만 Patch LLM에 전달
 - 고정 schema를 prompt 앞부분에 두어 prompt cache 활용
 - selector가 불확실하면 더 넓은 schema 또는 전체 schema로 fallback
-

4. Validation Gate

질문: Validation은 LLM 기반인가? 기존 Hybrid에도 있는가?

현재 방식:

Tool-Schema Patch의 Validation Gate는 규칙 기반 코드로 동작한다.

- domain/topic이 Tool schema에 존재하는지
- value가 타입·enum·범위에 맞는지
- evidence quote가 원문에 존재하는지
- evidence의 start/end 위치가 정확한지
- UPDATE 대상 record가 존재하는지
- UPDATE 근거에 변경 표현이 있는지
- 동일 identity/value가 이미 존재하는지
- user/scope 권한과 PII 정책을 지키는지

기존 Hybrid에는 동일한 Patch Gate가 없다. 일반적인 중복·PII·candidate 검증은 있지만 VehicleMem 기존 평가에서는 명시적 validation gate가 비활성화되어 있었다.

개선 방향:

엄격한 검사와 유연한 검사를 분리한다.

엄격하게 유지:

- JSON 구조와 타입
- Tool argument 범위
- 사용자·workspace 경계

- 대상 record 존재 여부
- transaction과 idempotency
- PII·보안 정책

유연하게 변경:

- 이름과 alias 비교
- evidence 위치의 공백·구두점 차이
- 자연어 변경 표현
- evidence와 value의 의미적 일치

확신이 낮은 Patch는 바로 거절하기보다 `review` 또는 별도 후보로 저장한다. 재시도 시에는 이전 거절 원인을 LLM에 피드백하고, 반복 실패는 안전한 NO-OP으로 종료한다.

5. SQLite Memory Record 식별

질문: 어떤 기준으로 새 record와 UPDATE를 구분하는가?

현재 identity 기준:

```
user_id
+ domain
+ topic
+ scope
+ scope_key
+ conditions
```

이 중 하나라도 다르면 별도 record가 된다. 따라서 다음 값들이 서로 다른 record가 될 수 있다.

```
person=Patricia
person=Patricia Garcia
person=Patricia, time=daytime
```

개선 방향:

Memory identity와 적용 조건을 분리한다.

고정 identity:

user/entity + domain + topic + 대상 좌석·기기

적용 조건:

시간 + 날씨 + 장소 + 운전자/탑승자 역할 + 기타 상황

판단 규칙:

- 이름은 canonical `entity_id` 와 alias로 정규화
- 같은 identity와 호환되는 조건이면 UPDATE 후보
- 더 구체적인 조건이면 별도 conditional record
- 조건 범위가 겹치면서 값이 다르면 conflict/version 처리
- 불확실한 alias는 자동 병합하지 않고 후보 상태로 저장

전체 구조는 엄격하게 유지하되, `conditions` 와 자연어 Memory 내용은 유연하게 저장한다.

6. Hybrid와 Patch 저장 방식의 차이

Structured Hybrid:

- LLM이 일반적인 사실·선호·상태를 구조화해 저장
- Tool domain/topic에 강제로 연결하지 않음
- 일반적인 중복 제거와 의미 검색에 유리
- 명시적인 변경·삭제·버전 추적은 상대적으로 약함

Patch:

- Tool domain/topic과 기존 record를 기준으로 연산 수행
- UPDATE·MERGE·DELETE와 source turn을 추적할 수 있음
- Tool 연결은 명확하지만 잘못 분류하면 검색 단계에서 누락될 수 있음

개선 방향은 Hybrid의 유연한 의미 표현과 Patch의 변경 추적 구조를 결합하는 것이다.

7. Tool Routing

질문: 단순 keyword routing 때문에 의미상 관련된 Tool을 놓치는가?

현재 방식:

- Tool 이름·domain·topic·argument와 질문의 keyword를 비교한다.
- 선택된 domain을 hard filter로 사용한다.
- 잘못된 domain이 선택되면 올바른 Memory는 embedding 검색에도 참여하지 못한다.

개선 방향:

Keyword 점수
+ Tool-description embedding 점수
+ 선택적 LLM classifier/reranker

Keyword 결과는 hard filter가 아니라 하나의 점수로만 사용한다. Recall 중심으로 여러 Tool 후보를 남기고 의미 기반으로 최종 정렬한다.

8. Memory Retrieval

현재 검색 과정:

1. 질문과 관련 있어 보이는 Tool/domain 선택
2. 사용자·인물·시간·상황이 맞는 Memory만 남김
3. BM25 keyword 유사도와 embedding 의미 유사도 계산
4. 가장 높은 Top-k Memory 선택

문제점:

- Tool routing이나 조건 필터에서 틀리면 정답 Memory가 검색 전에 제거됨
- 이름 alias와 역할 표현이 다르면 condition mismatch 발생
- embedding은 이미 좁혀진 후보 안에서만 작동함

개선 방향:

- 넓은 Tool 후보군을 먼저 생성
- 사용자 경계만 엄격하게 적용
- alias·시간·상황 호환성은 점수로 반영
- BM25·embedding·condition compatibility를 함께 계산
- reranker 이후 Top-k 선택
- 충돌하는 Memory가 있으면 최신성·근거·조건 특이성을 비교

9. Agent Context와 Memory 근거

Agent에게 제공되는 Context:

- 시스템 지침
- 현재 질문 전체
- 선택된 Memory
- 로드된 Tool schema
- Tool 실행 결과

제공하지 않는 것:

- 전체 장기 대화 원문
- 선택되지 않은 Memory
- 불필요한 Patch trace

Memory 원본 위치:

현재도 각 record에 source turn과 evidence quote가 별도로 저장된다. 따라서 어떤 대화에서 생성된 Memory인지 추적할 수 있다.

개선 방향:

- Agent에게는 compact Memory만 전달
- source turn과 evidence는 DB trace로 항상 보존
- 디버깅·검증 시 원문과 자동 대조
- 필요할 때만 verifier가 evidence를 읽도록 분리
- 일반 Agent Context에는 원문 전체를 넣지 않아 token과 개인정보 노출을 제한

검증 순서

개선 과정에서는 다음 실험으로 오류 위치를 분리한다.

1. Gold Tool + Gold Memory: Agent 실행 성능
2. Gold Tool + Patch Memory: Memory 추출·저장 성능
3. Tool Selector + Gold Memory: Routing·Retrieval 성능
4. Tool Selector + Patch Memory: 최종 E2E 성능

Gold 정보를 사용한 결과는 진단용 상한선으로만 사용하고, 최종 성능에는 포함하지 않는다.